State-of-the-Art Algorithmic Trading Model Implementation Guide

Optimized for LLM Implementation (2024-2026)

EXECUTIVE SUMMARY

Current State-of-the-Art Approaches (2024-2026):

1. **Temporal Fusion Transformer (TFT)** - Best for multi-horizon forecasting with interpretability
2. **Transformer-Based Architectures** (Pyraformer, Informer, Dual-Attention Transformers) - Superior long-range dependency modeling
3. **Deep Reinforcement Learning** (DQN, DDQN, Dueling DDQN, PPO) - Optimal for dynamic decision-making
4. **Hybrid Approaches** - Combining transformers for prediction + DRL for trading decisions

Key Finding: Hybrid frameworks combining LSTM/Transformer predictions with DRL trading agents consistently outperform single-approach methods across recent literature (2024-2025).

PART 1: PROBLEM FORMULATION

1.1 Task Types

PRIMARY TASK OPTIONS:

Option A: Price/Return Forecasting (Regression)

- **Output:** Continuous value (next price, % return, volatility)
- **Horizon:** Single-step (t+1) or multi-step (t+1 to t+n)
- **Use Case:** Feed predictions into rule-based or RL trading system
- **Metrics:** MAE, MSE, RMSE, MAPE, Directional Accuracy

Option B: Directional Classification

- **Output:** Discrete classes (UP/DOWN/HOLD or multi-class price change bins)
- **Use Case:** Simpler but may lose granularity
- **Metrics:** Accuracy, F1-score, Precision/Recall, Confusion Matrix

Option C: Direct Policy Learning (Reinforcement Learning)

- **Output:** Trading actions (BUY/SELL/HOLD or continuous position sizes)
- **State:** Market features + portfolio state
- **Action Space:** Discrete {Buy, Sell, Hold} or Continuous [position_size]
- **Reward:** Portfolio returns, Sharpe ratio, profit-adjusted metrics
- **Metrics:** Cumulative Return, Sharpe Ratio, Max Drawdown, Win Rate

RECOMMENDED APPROACH (State-of-the-Art 2024-2026): Two-Stage Hybrid Framework:

1. **Stage 1:** Transformer-based forecasting model for feature extraction
2. **Stage 2:** Deep RL agent for trading decisions using forecasts as state features

---

1.2 Data Characteristics & Challenges

Your Data:

- Hourly OHLCV (Open, High, Low, Close, Volume)
- Multiple assets (equities + cryptos)
- Missing occasional dates (irregular sampling)

Key Challenges:

1. **Non-stationarity:** Price distributions shift over time
2. **High noise-to-signal ratio:** Financial markets are noisy
3. **Missing data:** Gaps in hourly series
4. **Regime changes:** Market behavior shifts (bull/bear/crisis)
5. **Multi-asset correlation:** Cross-asset dependencies
6. **Transaction costs:** Slippage, fees impact profitability

---

PART 2: DATA PREPROCESSING

2.1 Missing Data Handling

STRATEGIES (in order of preference):

```
python
```

```python
# Priority 1: Forward fill for short gaps (< 3 hours)
df['close'] = df.groupby('ticker')['close'].fillna(method='ffill', limit=3)

# Priority 2: Interpolation for medium gaps (3-24 hours)
df['close'] = df.groupby('ticker')['close'].interpolate(method='linear', limit=24)

# Priority 3: Drop or flag assets with excessive missing data (> 5% of total)
missing_pct = df.groupby('ticker')['close'].apply(lambda x: x.isna().sum() / len(x))
valid_tickers = missing_pct[missing_pct < 0.05].index

# Priority 4: Add binary missingness indicator as feature
df['was_missing'] = df['close'].isna().astype(int)
```

CRITICAL: Never forward-fill into the future during training (data leakage).

2.2 Stationarity Transformation

REQUIRED TRANSFORMATIONS:

```python
# 1. Log returns (preferred over raw prices)
df['log_return'] = np.log(df['close'] / df['close'].shift(1))

# 2. Percentage returns (alternative)
df['pct_return'] = df['close'].pct_change()

# 3. Price differencing (if using raw prices)
df['price_diff'] = df['close'].diff()

# 4. Detrending via moving average
df['detrended_price'] = df['close'] - df['close'].rolling(24).mean()
```

WHY: Transformers and neural networks perform better on stationary data. Returns are more stationary than raw prices.

2.3 Normalization/Scaling

CRITICAL RULES:

1. **Fit scaler ONLY on training data**

2. **Apply same scaler to validation/test**
3. **Use rolling window normalization for time-series**

RECOMMENDED APPROACH:

```python
from sklearn.preprocessing import RobustScaler

# Option A: Per-asset rolling z-score (recommended)
def rolling_zscore(series, window=168):  # 168 hours = 1 week
    """Rolling z-score with expanding window for early periods."""
    rolling_mean = series.rolling(window, min_periods=24).mean()
    rolling_std = series.rolling(window, min_periods=24).std()
    return (series - rolling_mean) / (rolling_std + 1e-8)

df['close_norm'] = df.groupby('ticker')['close'].transform(rolling_zscore)

# Option B: RobustScaler (resistant to outliers)
scaler = RobustScaler()
train_data_scaled = scaler.fit_transform(train_data)
val_data_scaled = scaler.transform(val_data)  # Use same scaler

# Option C: Min-Max scaling (0-1 range)
from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler(feature_range=(-1, 1))  # Or (0, 1)
```

FOR MULTIPLE ASSETS:

- Normalize each asset independently OR
- Use rank-based normalization (cross-sectional z-score)

PART 3: FEATURE ENGINEERING

3.1 Technical Indicators (Essential)

MOMENTUM INDICATORS:

```python
```

```python
import ta  # Technical Analysis library

# 1. Relative Strength Index (RSI)
df['rsi_14'] = ta.momentum.RSIIndicator(df['close'], window=14).rsi()

# 2. Moving Average Convergence Divergence (MACD)
macd = ta.trend.MACD(df['close'])
df['macd'] = macd.macd()
df['macd_signal'] = macd.macd_signal()
df['macd_diff'] = macd.macd_diff()

# 3. Stochastic Oscillator
stoch = ta.momentum.StochasticOscillator(df['high'], df['low'], df['close'])
df['stoch_k'] = stoch.stoch()
df['stoch_d'] = stoch.stoch_signal()

# 4. Rate of Change (ROC)
df['roc_12'] = ta.momentum.ROCIndicator(df['close'], window=12).roc()
```

TREND INDICATORS:

```python
# 1. Moving Averages
df['sma_24'] = ta.trend.SMAIndicator(df['close'], window=24).sma_indicator()
df['ema_24'] = ta.trend.EMAIndicator(df['close'], window=24).ema_indicator()
df['sma_168'] = ta.trend.SMAIndicator(df['close'], window=168).sma_indicator()

# 2. Bollinger Bands
bb = ta.volatility.BollingerBands(df['close'], window=24, window_dev=2)
df['bb_high'] = bb.bollinger_hband()
df['bb_low'] = bb.bollinger_lband()
df['bb_mid'] = bb.bollinger_mavg()
df['bb_width'] = (df['bb_high'] - df['bb_low']) / df['bb_mid']

# 3. Average Directional Index (ADX)
df['adx'] = ta.trend.ADXIndicator(df['high'], df['low'], df['close'], window=14).adx

# 4. Supertrend (from recent research)
# Multiple parameter combinations: (12,3), (11,2), (10,1)
# Implementation via ta library or custom function
```

VOLATILITY INDICATORS:

python

```python
# 1. Average True Range (ATR)
df['atr'] = ta.volatility.AverageTrueRange(df['high'], df['low'], df['close'], windo

# 2. Historical Volatility
df['volatility_24h'] = df['log_return'].rolling(24).std()
df['volatility_168h'] = df['log_return'].rolling(168).std()

# 3. Parkinson's Volatility (uses high-low range)
df['parkinson_vol'] = np.sqrt(1/(4*np.log(2)) * (np.log(df['high']/df['low']))**2)
```

VOLUME INDICATORS:

python

```python
# 1. On-Balance Volume (OBV)
df['obv'] = ta.volume.OnBalanceVolumeIndicator(df['close'], df['volume']).on_balance

# 2. Volume-Weighted Average Price (VWAP)
df['vwap'] = (df['volume'] * (df['high'] + df['low'] + df['close']) / 3).cumsum() /

# 3. Money Flow Index (MFI)
df['mfi'] = ta.volume.MFIIndicator(df['high'], df['low'], df['close'], df['volume'],

# 4. Volume rate of change
df['volume_roc'] = df['volume'].pct_change(periods=24)
```

3.2 Time-Based Features

python

```python
# Cyclical encoding (important for neural networks)
df['hour'] = df.index.hour
df['hour_sin'] = np.sin(2 * np.pi * df['hour'] / 24)
df['hour_cos'] = np.cos(2 * np.pi * df['hour'] / 24)

df['day_of_week'] = df.index.dayofweek
df['dow_sin'] = np.sin(2 * np.pi * df['day_of_week'] / 7)
df['dow_cos'] = np.cos(2 * np.pi * df['day_of_week'] / 7)

df['day_of_month'] = df.index.day
df['dom_sin'] = np.sin(2 * np.pi * df['day_of_month'] / 31)
df['dom_cos'] = np.cos(2 * np.pi * df['day_of_month'] / 31)

# Market regime indicators
df['is_market_open'] = ((df.index.hour >= 9) & (df.index.hour < 16)).astype(int)  #
df['is_weekend'] = (df.index.dayofweek >= 5).astype(int)
```

3.3 Lagged Features

python

```python
# Create multiple lag windows
lags = [1, 2, 3, 6, 12, 24, 168]  # 1h, 2h, 3h, 6h, 12h, 24h, 1 week
for lag in lags:
    df[f'return_lag_{lag}'] = df['log_return'].shift(lag)
    df[f'volume_lag_{lag}'] = df['volume'].shift(lag)
    df[f'volatility_lag_{lag}'] = df['volatility_24h'].shift(lag)
```

3.4 Rolling Statistical Features

python

```python
windows = [6, 24, 168]  # 6h, 24h, 1 week

for window in windows:
    # Returns statistics
    df[f'return_mean_{window}h'] = df['log_return'].rolling(window).mean()
    df[f'return_std_{window}h'] = df['log_return'].rolling(window).std()
    df[f'return_skew_{window}h'] = df['log_return'].rolling(window).skew()
    df[f'return_kurt_{window}h'] = df['log_return'].rolling(window).kurt()

    # Price statistics
    df[f'price_max_{window}h'] = df['close'].rolling(window).max()
    df[f'price_min_{window}h'] = df['close'].rolling(window).min()
    df[f'price_range_{window}h'] = df[f'price_max_{window}h'] - df[f'price_min_{wind

    # Volume statistics
    df[f'volume_mean_{window}h'] = df['volume'].rolling(window).mean()
    df[f'volume_std_{window}h'] = df['volume'].rolling(window).std()
```

3.5 Cross-Asset Features (Multi-Asset Portfolio)

python

```python
# Correlation with market index or Bitcoin (for crypto)
# Assuming 'BTC' or 'SPY' as benchmark
benchmark_returns = df[df['ticker'] == 'BTC']['log_return']

for ticker in df['ticker'].unique():
    if ticker != 'BTC':
        ticker_returns = df[df['ticker'] == ticker]['log_return']
        df.loc[df['ticker'] == ticker, 'correlation_to_btc'] = \
            ticker_returns.rolling(168).corr(benchmark_returns)

# Relative strength vs benchmark
df['relative_strength'] = df['log_return'] - benchmark_returns

# Cross-sectional rank (percentile among all assets)
df['rank_return'] = df.groupby(df.index)['log_return'].rank(pct=True)
df['rank_volume'] = df.groupby(df.index)['volume'].rank(pct=True)
```

3.6 Crypto-Specific Features (If Applicable)

ON-CHAIN METRICS (requires external data sources):

```python
# If available from APIs like Glassnode, CryptoQuant:
# - Active Addresses (AA)
# - Exchange Net Flow (ENF)
# - Spent Output Profit Ratio (SOPR)
# - Realized Cap HODL Waves
# - Hash Rate
# - Mining Difficulty
# - Total Value Locked (TVL) for DeFi tokens

# Sentiment Features
# - Crypto Fear & Greed Index
# - Social media sentiment scores
# - News sentiment (from NLP on headlines)
```

3.7 Feature Selection & Dimensionality Reduction

AFTER CREATING FEATURES:

```python
```

```python
from sklearn.feature_selection import mutual_info_regression, SelectKBest
from sklearn.ensemble import RandomForestRegressor

# Method 1: Mutual Information (for non-linear relationships)
selector = SelectKBest(score_func=mutual_info_regression, k=50)
X_selected = selector.fit_transform(X_train, y_train)
selected_features = X_train.columns[selector.get_support()]

# Method 2: Tree-based feature importance
rf = RandomForestRegressor(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)
importances = pd.DataFrame({
    'feature': X_train.columns,
    'importance': rf.feature_importances_
}).sort_values('importance', ascending=False)
top_features = importances.head(50)['feature'].tolist()

# Method 3: Recursive Feature Elimination
from sklearn.feature_selection import RFE
estimator = RandomForestRegressor(n_estimators=50)
rfe = RFE(estimator, n_features_to_select=50, step=1)
rfe.fit(X_train, y_train)
selected_features = X_train.columns[rfe.support_]

# Method 4: L1 Regularization (LASSO)
from sklearn.linear_model import LassoCV
lasso = LassoCV(cv=5, random_state=42, max_iter=10000)
lasso.fit(X_train, y_train)
selected_features = X_train.columns[np.abs(lasso.coef_) > 1e-5]
```

RECOMMENDATION: Start with 30-100 most important features. Too many features can cause overfitting.

PART 4: MODEL ARCHITECTURES

4.1 Temporal Fusion Transformer (TFT) - SOTA for Multi-Horizon Forecasting

ARCHITECTURE OVERVIEW:

- **Input:** Static features (asset ID) + time-varying features (technical indicators, OHLCV)
- **Components:**

1. Variable Selection Networks (gate which features to use)
2. LSTM Encoders for historical context
3. Multi-Head Attention for temporal dependencies
4. Quantile Regression outputs for uncertainty

IMPLEMENTATION (PyTorch Lightning):

```python
```

```python
from pytorch_forecasting import TemporalFusionTransformer, TimeSeriesDataSet
from pytorch_forecasting.data import GroupNormalizer
from pytorch_lightning.callbacks import EarlyStopping

# Prepare dataset
training = TimeSeriesDataSet(
    df[lambda x: x.time_idx < cutoff],
    time_idx="time_idx",
    target="log_return",
    group_ids=["ticker"],
    min_encoder_length=168,  # 1 week history
    max_encoder_length=168,
    min_prediction_length=1,
    max_prediction_length=24,  # Forecast next 24 hours
    static_categoricals=["ticker"],
    time_varying_known_reals=["hour_sin", "hour_cos", "dow_sin", "dow_cos"],
    time_varying_unknown_reals=[
        "log_return", "close", "volume", "rsi_14", "macd",
        "atr", "bb_width", "volatility_24h", "obv"
    ],
    target_normalizer=GroupNormalizer(
        groups=["ticker"], transformation="softplus"
    ),
    add_relative_time_idx=True,
    add_target_scales=True,
    add_encoder_length=True,
)

# TFT Model
tft = TemporalFusionTransformer.from_dataset(
    training,
    learning_rate=1e-3,
    hidden_size=64,  # Reduce if memory-constrained
    attention_head_size=4,
    dropout=0.1,
    hidden_continuous_size=16,
    output_size=7,  # 7 quantiles for uncertainty
    loss=QuantileLoss(),
    log_interval=100,
    reduce_on_plateau_patience=4,
)

# Train
```

```
trainer = pl.Trainer(
    max_epochs=50,
    gpus=1,
    gradient_clip_val=0.1,
    callbacks=[EarlyStopping(monitor="val_loss", patience=5)],
)
trainer.fit(tft, train_dataloader=train_dataloader, val_dataloaders=val_dataloader)
```

HYPERPARAMETERS:

- `hidden_size`: 32-128 (64 recommended)
- `attention_head_size`: 1-4
- `dropout`: 0.1-0.3
- `learning_rate`: 1e-4 to 1e-2 (use scheduler)

4.2 Transformer Variants

INFORMER (for very long sequences):

- Handles sequences > 1000 steps efficiently
- ProbSparse self-attention mechanism
- Use when you have > 1 month of hourly data per window

PYRAFORMER (best performance in recent studies):

- Pyramidal attention structure
- Low computational complexity
- Superior predictive accuracy on crypto data

IMPLEMENTATION STRUCTURE:

```
python
```

```
import torch
import torch.nn as nn

class PyraformerModel(nn.Module):
    def __init__(self, input_dim, d_model=64, nhead=4, num_layers=3, dropout=0.1):
        super().__init__()
        self.input_projection = nn.Linear(input_dim, d_model)

        # Pyramidal attention layers
        encoder_layer = nn.TransformerEncoderLayer(
            d_model=d_model,
            nhead=nhead,
            dropout=dropout,
            batch_first=True
        )
        self.transformer_encoder = nn.TransformerEncoder(encoder_layer, num_layers=n

        # Output layers
        self.fc_out = nn.Linear(d_model, 1)

    def forward(self, x):
        # x shape: [batch_size, seq_len, input_dim]
        x = self.input_projection(x)
        x = self.transformer_encoder(x)
        # Use last time step for prediction
        output = self.fc_out(x[:, -1, :])
        return output

# Training loop
model = PyraformerModel(input_dim=num_features, d_model=64, nhead=4, num_layers=3)
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
criterion = nn.MSELoss()

for epoch in range(num_epochs):
    for batch in train_loader:
        X_batch, y_batch = batch
        optimizer.zero_grad()
        predictions = model(X_batch)
        loss = criterion(predictions, y_batch)
        loss.backward()
        optimizer.step()
```

4.3 Hybrid LSTM-Transformer

ARCHITECTURE:

1. LSTM layers capture local patterns
2. Transformer layers capture global dependencies
3. Combines strengths of both

```python
class LSTMTransformer(nn.Module):
    def __init__(self, input_dim, lstm_hidden=64, d_model=64, nhead=4, num_transform
        super().__init__()

        # LSTM for local patterns
        self.lstm = nn.LSTM(input_dim, lstm_hidden, batch_first=True, num_layers=2)

        # Transformer for global patterns
        self.input_proj = nn.Linear(lstm_hidden, d_model)
        encoder_layer = nn.TransformerEncoderLayer(d_model, nhead, batch_first=True)
        self.transformer = nn.TransformerEncoder(encoder_layer, num_transformer_laye

        # Output
        self.fc = nn.Linear(d_model, 1)

    def forward(self, x):
        # LSTM
        lstm_out, _ = self.lstm(x)

        # Project to transformer dimension
        transformer_input = self.input_proj(lstm_out)

        # Transformer
        transformer_out = self.transformer(transformer_input)

        # Final prediction
        output = self.fc(transformer_out[:, -1, :])
        return output
```

4.4 Deep Reinforcement Learning Architectures

DQN (Deep Q-Network) - Baseline:

```python
```

```
import torch.nn as nn
import numpy as np
from collections import deque
import random

class DQN(nn.Module):
    def __init__(self, state_dim, action_dim=3, hidden_dim=128):
        super(DQN, self).__init__()
        self.fc1 = nn.Linear(state_dim, hidden_dim)
        self.fc2 = nn.Linear(hidden_dim, hidden_dim)
        self.fc3 = nn.Linear(hidden_dim, action_dim)  # BUY, SELL, HOLD

    def forward(self, x):
        x = torch.relu(self.fc1(x))
        x = torch.relu(self.fc2(x))
        return self.fc3(x)

class DQNAgent:
    def __init__(self, state_dim, action_dim=3, lr=1e-3, gamma=0.99):
        self.action_dim = action_dim
        self.gamma = gamma
        self.epsilon = 1.0
        self.epsilon_min = 0.01
        self.epsilon_decay = 0.995

        # Networks
        self.policy_net = DQN(state_dim, action_dim)
        self.target_net = DQN(state_dim, action_dim)
        self.target_net.load_state_dict(self.policy_net.state_dict())

        self.optimizer = torch.optim.Adam(self.policy_net.parameters(), lr=lr)
        self.memory = deque(maxlen=10000)

    def select_action(self, state):
        if np.random.rand() < self.epsilon:
            return np.random.randint(self.action_dim)
        with torch.no_grad():
            state_tensor = torch.FloatTensor(state).unsqueeze(0)
            q_values = self.policy_net(state_tensor)
            return q_values.argmax().item()

    def store_transition(self, state, action, reward, next_state, done):
        self.memory.append((state, action, reward, next_state, done))
```

```
    def train_step(self, batch_size=64):
        if len(self.memory) < batch_size:
            return

        batch = random.sample(self.memory, batch_size)
        states, actions, rewards, next_states, dones = zip(*batch)

        states = torch.FloatTensor(states)
        actions = torch.LongTensor(actions)
        rewards = torch.FloatTensor(rewards)
        next_states = torch.FloatTensor(next_states)
        dones = torch.FloatTensor(dones)

        # Q-learning update
        current_q = self.policy_net(states).gather(1, actions.unsqueeze(1))
        next_q = self.target_net(next_states).max(1)[0].detach()
        target_q = rewards + (1 - dones) * self.gamma * next_q

        loss = nn.MSELoss()(current_q.squeeze(), target_q)

        self.optimizer.zero_grad()
        loss.backward()
        self.optimizer.step()

        # Decay epsilon
        self.epsilon = max(self.epsilon_min, self.epsilon * self.epsilon_decay)
```

DOUBLE DQN (DDQN) - Improved:

python

```
# Modification in train_step:
def train_step_ddqn(self, batch_size=64):
    # ... (same setup as DQN)

    # Double DQN: use policy_net to select action, target_net to evaluate
    next_actions = self.policy_net(next_states).argmax(1)
    next_q = self.target_net(next_states).gather(1, next_actions.unsqueeze(1)).squee
    target_q = rewards + (1 - dones) * self.gamma * next_q

    # ... (same loss computation)
```

DUELING DDQN (State-of-the-Art for Trading):

python

```python
class DuelingDQN(nn.Module):
    def __init__(self, state_dim, action_dim=3, hidden_dim=128):
        super(DuelingDQN, self).__init__()

        # Shared feature extractor
        self.feature = nn.Sequential(
            nn.Linear(state_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, hidden_dim),
            nn.ReLU()
        )

        # Value stream
        self.value_stream = nn.Sequential(
            nn.Linear(hidden_dim, hidden_dim // 2),
            nn.ReLU(),
            nn.Linear(hidden_dim // 2, 1)
        )

        # Advantage stream
        self.advantage_stream = nn.Sequential(
            nn.Linear(hidden_dim, hidden_dim // 2),
            nn.ReLU(),
            nn.Linear(hidden_dim // 2, action_dim)
        )

    def forward(self, x):
        features = self.feature(x)
        value = self.value_stream(features)
        advantages = self.advantage_stream(features)

        # Q(s,a) = V(s) + (A(s,a) - mean(A(s,a)))
        q_values = value + (advantages - advantages.mean(dim=1, keepdim=True))
        return q_values
```

PPO (Proximal Policy Optimization) - Best for Continuous Actions:

python

```
import torch
import torch.nn as nn
from torch.distributions import Normal

class ActorCritic(nn.Module):
    def __init__(self, state_dim, action_dim=1, hidden_dim=128):
        super(ActorCritic, self).__init__()

        # Actor (policy) network
        self.actor = nn.Sequential(
            nn.Linear(state_dim, hidden_dim),
            nn.Tanh(),
            nn.Linear(hidden_dim, hidden_dim),
            nn.Tanh(),
            nn.Linear(hidden_dim, action_dim)
        )

        # Critic (value) network
        self.critic = nn.Sequential(
            nn.Linear(state_dim, hidden_dim),
            nn.Tanh(),
            nn.Linear(hidden_dim, hidden_dim),
            nn.Tanh(),
            nn.Linear(hidden_dim, 1)
        )

        # Log standard deviation for action distribution
        self.log_std = nn.Parameter(torch.zeros(action_dim))

    def forward(self, state):
        return self.actor(state), self.critic(state)

    def get_action(self, state):
        mean = self.actor(state)
        std = self.log_std.exp()
        dist = Normal(mean, std)
        action = dist.sample()
        log_prob = dist.log_prob(action)
        return action, log_prob

    def evaluate(self, state, action):
        mean = self.actor(state)
        std = self.log_std.exp()
```

```
        dist = Normal(mean, std)
        log_prob = dist.log_prob(action)
        entropy = dist.entropy()
        value = self.critic(state)
        return log_prob, value, entropy

class PPOAgent:
    def __init__(self, state_dim, action_dim=1, lr=3e-4, gamma=0.99,
                 clip_epsilon=0.2, epochs=10):
        self.gamma = gamma
        self.clip_epsilon = clip_epsilon
        self.epochs = epochs

        self.policy = ActorCritic(state_dim, action_dim)
        self.optimizer = torch.optim.Adam(self.policy.parameters(), lr=lr)

    def update(self, states, actions, old_log_probs, returns, advantages):
        for _ in range(self.epochs):
            log_probs, values, entropy = self.policy.evaluate(states, actions)

            # Ratio for clipping
            ratio = torch.exp(log_probs - old_log_probs)

            # Clipped surrogate loss
            surr1 = ratio * advantages
            surr2 = torch.clamp(ratio, 1 - self.clip_epsilon, 1 + self.clip_epsilon)
            actor_loss = -torch.min(surr1, surr2).mean()

            # Value loss
            critic_loss = nn.MSELoss()(values.squeeze(), returns)

            # Entropy bonus (encourages exploration)
            entropy_loss = -entropy.mean()

            # Total loss
            loss = actor_loss + 0.5 * critic_loss + 0.01 * entropy_loss

            self.optimizer.zero_grad()
            loss.backward()
            torch.nn.utils.clip_grad_norm_(self.policy.parameters(), 0.5)
            self.optimizer.step()
```

PART 5: LOSS FUNCTIONS

5.1 For Forecasting Models

REGRESSION LOSSES:

```python
```

```python
import torch.nn as nn

# 1. Mean Squared Error (MSE) - Standard baseline
loss_fn = nn.MSELoss()

# 2. Mean Absolute Error (MAE) - More robust to outliers
loss_fn = nn.L1Loss()

# 3. Huber Loss - Combines MSE and MAE benefits
loss_fn = nn.SmoothL1Loss()

# 4. Quantile Loss - For uncertainty estimation (TFT uses this)
class QuantileLoss(nn.Module):
    def __init__(self, quantiles=[0.1, 0.5, 0.9]):
        super().__init__()
        self.quantiles = quantiles

    def forward(self, predictions, targets):
        losses = []
        for i, q in enumerate(self.quantiles):
            errors = targets - predictions[:, i]
            losses.append(torch.max((q - 1) * errors, q * errors))
        return torch.mean(torch.sum(torch.cat(losses, dim=1), dim=1))

# 5. Custom: MSE + Directional Accuracy (Hybrid)
class HybridLoss(nn.Module):
    def __init__(self, alpha=0.7):
        super().__init__()
        self.alpha = alpha

    def forward(self, predictions, targets):
        # MSE component
        mse = torch.mean((predictions - targets) ** 2)

        # Directional accuracy component
        pred_direction = torch.sign(predictions)
        true_direction = torch.sign(targets)
        directional_accuracy = torch.mean((pred_direction == true_direction).float()
        directional_loss = 1 - directional_accuracy

        return self.alpha * mse + (1 - self.alpha) * directional_loss
```

CLASSIFICATION LOSSES:

python

```python
# 1. Cross-Entropy (for directional classification)
loss_fn = nn.CrossEntropyLoss()

# 2. Focal Loss - Handles class imbalance (e.g., rare price jumps)
class FocalLoss(nn.Module):
    def __init__(self, alpha=0.25, gamma=2):
        super().__init__()
        self.alpha = alpha
        self.gamma = gamma

    def forward(self, predictions, targets):
        ce_loss = nn.CrossEntropyLoss(reduction='none')(predictions, targets)
        p_t = torch.exp(-ce_loss)
        focal_loss = self.alpha * (1 - p_t) ** self.gamma * ce_loss
        return focal_loss.mean()
```

5.2 For Reinforcement Learning

REWARD FUNCTIONS (Critical for RL Success):

python

```python
# Option 1: Simple return-based reward
def calculate_reward_simple(action, price_change, position):
    """
    action: 0=SELL, 1=HOLD, 2=BUY
    price_change: next_price / current_price - 1
    position: current position (-1, 0, 1)
    """
    # BUY
    if action == 2:
        reward = price_change if price_change > 0 else 2 * price_change
    # SELL
    elif action == 0:
        reward = -price_change if price_change < 0 else 2 * price_change
    # HOLD
    else:
        reward = 0

    return reward

# Option 2: Portfolio value change (better)
def calculate_reward_portfolio(action, price_change, position, transaction_cost=0.00
    """
    Reward based on change in portfolio value.
    """
    next_position = position

    # Update position based on action
    if action == 2 and position != 1:  # BUY
        next_position = 1
        cost = transaction_cost
    elif action == 0 and position != -1:  # SELL
        next_position = -1
        cost = transaction_cost
    else:  # HOLD or no change
        cost = 0

    # Calculate PnL
    pnl = position * price_change - cost

    return pnl

# Option 3: Sharpe ratio-based (advanced)
def calculate_reward_sharpe(returns_history, window=100, risk_free_rate=0.0):
```

```
    """
    Rolling Sharpe ratio as reward signal.
    """
    if len(returns_history) < window:
        return 0.0

    recent_returns = returns_history[-window:]
    mean_return = np.mean(recent_returns)
    std_return = np.std(recent_returns)

    if std_return == 0:
        return 0.0

    sharpe = (mean_return - risk_free_rate) / std_return
    return sharpe

# Option 4: Multi-objective (SOTA from recent papers)
def calculate_reward_multi_objective(action, price_change, position,
                                     portfolio_value, max_drawdown,
                                     transaction_cost=0.001):
    """
    Combines multiple objectives:
    - Profit
    - Risk (drawdown penalty)
    - Transaction cost
    """
    # PnL component
    pnl = position * price_change

    # Transaction cost
    if (action == 2 and position != 1) or (action == 0 and position != -1):
        cost = transaction_cost
    else:
        cost = 0

    # Drawdown penalty
    drawdown_penalty = -0.1 * max(0, max_drawdown - 0.1)  # Penalty if drawdown > 10

    # Combined reward
    reward = pnl - cost + drawdown_penalty

    return reward
```

RECOMMENDED: Use Option 2 (portfolio value change) as baseline, Option 4 for best results.

PART 6: TRAINING STRATEGY

6.1 Train-Validation-Test Split

CRITICAL: Use Walk-Forward or Expanding Window (NOT Random Split)

```python
```

```python
# Option A: Walk-Forward Validation (Recommended)
def walk_forward_split(df, n_splits=5):
    """
    Split data into sequential folds for time-series cross-validation.
    """
    from sklearn.model_selection import TimeSeriesSplit

    tscv = TimeSeriesSplit(n_splits=n_splits)
    splits = []

    for train_idx, val_idx in tscv.split(df):
        train_data = df.iloc[train_idx]
        val_data = df.iloc[val_idx]
        splits.append((train_data, val_data))

    return splits

# Option B: Fixed Expanding Window
def expanding_window_split(df, train_pct=0.6, val_pct=0.2):
    """
    60% train, 20% validation, 20% test.
    """
    n = len(df)
    train_end = int(n * train_pct)
    val_end = int(n * (train_pct + val_pct))

    train_data = df.iloc[:train_end]
    val_data = df.iloc[train_end:val_end]
    test_data = df.iloc[val_end:]

    return train_data, val_data, test_data

# Example usage
train_df, val_df, test_df = expanding_window_split(df)
```

IMPORTANT:

- Never shuffle time-series data
- Test set must be the most recent data
- Validation set used for early stopping

6.2 Handling Data Leakage

COMMON PITFALLS:

```python
# ❌ WRONG: Calculating statistics on entire dataset
scaler = StandardScaler()
df['normalized'] = scaler.fit_transform(df[['close']])  # LEAKAGE!

# ✅ CORRECT: Fit only on training data
scaler = StandardScaler()
train_df['normalized'] = scaler.fit_transform(train_df[['close']])
val_df['normalized'] = scaler.transform(val_df[['close']])
test_df['normalized'] = scaler.transform(test_df[['close']])

# ❌ WRONG: Using future data in features
df['rsi'] = calculate_rsi(df['close'])  # If this looks ahead, LEAKAGE!

# ✅ CORRECT: Ensure all features use only past data
df['rsi'] = calculate_rsi(df['close'])  # RSI correctly uses only past prices
# Double-check no shift(-1) or future indexing

# ❌ WRONG: Calculating target using future normalization
df['target'] = (df['close'].shift(-1) - df['close']) / df['close']
# If 'close' was normalized on full dataset, this is LEAKAGE

# ✅ CORRECT: Calculate target before normalization, or use proper splits
```

6.3 Training Configuration

FORECASTING MODEL TRAINING:

```python
```

```
import torch
from torch.utils.data import DataLoader, TensorDataset

# Hyperparameters
SEQUENCE_LENGTH = 168  # 1 week of hourly data
BATCH_SIZE = 64
LEARNING_RATE = 1e-3
NUM_EPOCHS = 100
EARLY_STOPPING_PATIENCE = 10

# Data preparation
def create_sequences(data, seq_length):
    X, y = [], []
    for i in range(len(data) - seq_length):
        X.append(data[i:i+seq_length])
        y.append(data[i+seq_length, target_col_idx])  # Next time step
    return np.array(X), np.array(y)

X_train, y_train = create_sequences(train_data, SEQUENCE_LENGTH)
X_val, y_val = create_sequences(val_data, SEQUENCE_LENGTH)

# Convert to PyTorch tensors
X_train_tensor = torch.FloatTensor(X_train)
y_train_tensor = torch.FloatTensor(y_train)
X_val_tensor = torch.FloatTensor(X_val)
y_val_tensor = torch.FloatTensor(y_val)

# DataLoaders
train_dataset = TensorDataset(X_train_tensor, y_train_tensor)
train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=False)  # No

val_dataset = TensorDataset(X_val_tensor, y_val_tensor)
val_loader = DataLoader(val_dataset, batch_size=BATCH_SIZE, shuffle=False)

# Training loop with early stopping
best_val_loss = float('inf')
patience_counter = 0

for epoch in range(NUM_EPOCHS):
    # Training
    model.train()
    train_loss = 0
    for X_batch, y_batch in train_loader:
```

```
        optimizer.zero_grad()
        predictions = model(X_batch)
        loss = criterion(predictions, y_batch)
        loss.backward()
        optimizer.step()
        train_loss += loss.item()

    # Validation
    model.eval()
    val_loss = 0
    with torch.no_grad():
        for X_batch, y_batch in val_loader:
            predictions = model(X_batch)
            loss = criterion(predictions, y_batch)
            val_loss += loss.item()

    train_loss /= len(train_loader)
    val_loss /= len(val_loader)

    print(f"Epoch {epoch+1}/{NUM_EPOCHS}, Train Loss: {train_loss:.6f}, Val Loss: {v

    # Early stopping
    if val_loss < best_val_loss:
        best_val_loss = val_loss
        patience_counter = 0
        torch.save(model.state_dict(), 'best_model.pth')
    else:
        patience_counter += 1
        if patience_counter >= EARLY_STOPPING_PATIENCE:
            print("Early stopping triggered")
            break

# Load best model
model.load_state_dict(torch.load('best_model.pth'))
```

REINFORCEMENT LEARNING TRAINING:

python

```python
# Trading environment
class TradingEnvironment:
    def __init__(self, data, initial_balance=10000, transaction_cost=0.001):
        self.data = data
        self.initial_balance = initial_balance
        self.transaction_cost = transaction_cost
        self.reset()

    def reset(self):
        self.current_step = 0
        self.balance = self.initial_balance
        self.position = 0  # -1: short, 0: neutral, 1: long
        self.portfolio_value = self.initial_balance
        return self._get_state()

    def _get_state(self):
        """Return current market features + portfolio state."""
        market_features = self.data[self.current_step]
        portfolio_features = np.array([
            self.balance / self.initial_balance,
            self.position,
            self.portfolio_value / self.initial_balance
        ])
        return np.concatenate([market_features, portfolio_features])

    def step(self, action):
        """
        action: 0=SELL, 1=HOLD, 2=BUY
        """
        current_price = self.data[self.current_step, price_col_idx]
        next_price = self.data[self.current_step + 1, price_col_idx]
        price_change = (next_price / current_price) - 1

        # Execute action
        old_position = self.position
        if action == 2 and self.position != 1:  # BUY
            self.position = 1
            self.balance -= self.balance * self.transaction_cost
        elif action == 0 and self.position != -1:  # SELL
            self.position = -1
            self.balance -= self.balance * self.transaction_cost

        # Calculate reward (portfolio value change)
```

```
        pnl = old_position * price_change * self.portfolio_value
        self.portfolio_value += pnl
        reward = pnl / self.initial_balance  # Normalized reward

        # Move to next step
        self.current_step += 1
        done = self.current_step >= len(self.data) - 1
        next_state = self._get_state() if not done else None

        return next_state, reward, done

# DQN Training
env = TradingEnvironment(train_data)
agent = DQNAgent(state_dim=state_dim, action_dim=3)

num_episodes = 1000
for episode in range(num_episodes):
    state = env.reset()
    total_reward = 0
    done = False

    while not done:
        action = agent.select_action(state)
        next_state, reward, done = env.step(action)
        agent.store_transition(state, action, reward, next_state, done)
        agent.train_step(batch_size=64)

        state = next_state
        total_reward += reward

    # Update target network every 10 episodes
    if episode % 10 == 0:
        agent.target_net.load_state_dict(agent.policy_net.state_dict())

    print(f"Episode {episode+1}, Total Reward: {total_reward:.4f}, Epsilon: {agent.e
```

PART 7: EVALUATION METRICS

7.1 Forecasting Metrics

python

```python
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np

# Regression metrics
mae = mean_absolute_error(y_true, y_pred)
mse = mean_squared_error(y_true, y_pred)
rmse = np.sqrt(mse)
mape = np.mean(np.abs((y_true - y_pred) / y_true)) * 100

# R-squared
r2 = r2_score(y_true, y_pred)

# Directional Accuracy (Critical for trading)
def directional_accuracy(y_true, y_pred):
    """Percentage of times we predicted the correct direction."""
    correct_direction = np.sign(y_true) == np.sign(y_pred)
    return np.mean(correct_direction) * 100

da = directional_accuracy(y_true, y_pred)

# Symmetric Mean Absolute Percentage Error (handles zero values better)
def smape(y_true, y_pred):
    denominator = (np.abs(y_true) + np.abs(y_pred)) / 2
    return np.mean(np.abs(y_true - y_pred) / denominator) * 100

smape_score = smape(y_true, y_pred)

print(f"MAE: {mae:.6f}")
print(f"RMSE: {rmse:.6f}")
print(f"MAPE: {mape:.2f}%")
print(f"R²: {r2:.4f}")
print(f"Directional Accuracy: {da:.2f}%")
print(f"SMAPE: {smape_score:.2f}%")
```

MOST IMPORTANT FOR TRADING: Directional Accuracy > RMSE/MAE

7.2 Trading Performance Metrics

python

```python
import pandas as pd
import numpy as np

def calculate_trading_metrics(portfolio_values, returns, risk_free_rate=0.0):
    """
    Comprehensive trading performance metrics.

    portfolio_values: array of portfolio values over time
    returns: array of period returns
    risk_free_rate: annual risk-free rate (e.g., 0.02 for 2%)
    """
    # Total Return
    total_return = (portfolio_values[-1] / portfolio_values[0] - 1) * 100

    # Annualized Return (assuming hourly data)
    n_hours = len(returns)
    n_years = n_hours / (365 * 24)
    annualized_return = ((portfolio_values[-1] / portfolio_values[0]) ** (1 / n_year

    # Sharpe Ratio (annualized)
    returns_std = np.std(returns)
    sharpe_ratio = (np.mean(returns) - risk_free_rate / (365 * 24)) / (returns_std +
    sharpe_ratio_annual = sharpe_ratio * np.sqrt(365 * 24)  # Annualize

    # Sortino Ratio (uses only downside deviation)
    downside_returns = returns[returns < 0]
    downside_std = np.std(downside_returns) if len(downside_returns) > 0 else 0
    sortino_ratio = (np.mean(returns) - risk_free_rate / (365 * 24)) / (downside_std
    sortino_ratio_annual = sortino_ratio * np.sqrt(365 * 24)

    # Maximum Drawdown
    cumulative = portfolio_values / portfolio_values[0]
    running_max = np.maximum.accumulate(cumulative)
    drawdown = (cumulative - running_max) / running_max
    max_drawdown = np.min(drawdown) * 100

    # Calmar Ratio
    calmar_ratio = annualized_return / abs(max_drawdown) if max_drawdown != 0 else 0

    # Win Rate
    winning_returns = returns[returns > 0]
    win_rate = (len(winning_returns) / len(returns)) * 100
```

```
    # Profit Factor
    gross_profit = np.sum(returns[returns > 0])
    gross_loss = abs(np.sum(returns[returns < 0]))
    profit_factor = gross_profit / gross_loss if gross_loss != 0 else np.inf

    # Average Win / Average Loss
    avg_win = np.mean(winning_returns) if len(winning_returns) > 0 else 0
    losing_returns = returns[returns < 0]
    avg_loss = np.mean(abs(losing_returns)) if len(losing_returns) > 0 else 0
    avg_win_loss_ratio = avg_win / avg_loss if avg_loss != 0 else np.inf

    # Volatility (annualized)
    volatility = np.std(returns) * np.sqrt(365 * 24) * 100

    metrics = {
        'Total Return (%)': total_return,
        'Annualized Return (%)': annualized_return,
        'Sharpe Ratio': sharpe_ratio_annual,
        'Sortino Ratio': sortino_ratio_annual,
        'Max Drawdown (%)': max_drawdown,
        'Calmar Ratio': calmar_ratio,
        'Win Rate (%)': win_rate,
        'Profit Factor': profit_factor,
        'Avg Win/Loss Ratio': avg_win_loss_ratio,
        'Volatility (%)': volatility
    }

    return metrics

# Example usage
metrics = calculate_trading_metrics(portfolio_values, returns)
for metric, value in metrics.items():
    print(f"{metric}: {value:.2f}")
```

KEY METRICS TO OPTIMIZE:

1. **Sharpe Ratio** > 1.0 (excellent), > 2.0 (exceptional)
2. **Max Drawdown** < 20% (good), < 10% (excellent)
3. **Win Rate** > 50% for balanced strategy
4. **Profit Factor** > 1.5

7.3 Backtesting Framework

python

```python
class Backtester:
    def __init__(self, model, data, initial_balance=10000, transaction_cost=0.001):
        self.model = model
        self.data = data
        self.initial_balance = initial_balance
        self.transaction_cost = transaction_cost

    def run(self):
        """Execute backtest and return detailed results."""
        balance = self.initial_balance
        position = 0
        portfolio_values = [self.initial_balance]
        returns = []
        trades = []

        for i in range(len(self.data) - 1):
            state = self._get_state(i)

            # Get model prediction/action
            if isinstance(self.model, DQNAgent):
                action = self.model.select_action(state)
            else:  # Forecasting model
                forecast = self.model.predict(state)
                action = self._forecast_to_action(forecast)

            # Execute trade
            current_price = self.data[i, price_col_idx]
            next_price = self.data[i + 1, price_col_idx]

            old_position = position
            if action == 2 and position != 1:  # BUY
                position = 1
                balance -= balance * self.transaction_cost
                trades.append({'time': i, 'action': 'BUY', 'price': current_price})
            elif action == 0 and position != -1:  # SELL
                position = -1
                balance -= balance * self.transaction_cost
                trades.append({'time': i, 'action': 'SELL', 'price': current_price})

            # Calculate return
            price_return = (next_price / current_price) - 1
            portfolio_return = old_position * price_return
            portfolio_value = portfolio_values[-1] * (1 + portfolio_return)
```

```
            portfolio_values.append(portfolio_value)
            returns.append(portfolio_return)

        # Calculate metrics
        metrics = calculate_trading_metrics(
            np.array(portfolio_values),
            np.array(returns)
        )

        results = {
            'metrics': metrics,
            'portfolio_values': portfolio_values,
            'returns': returns,
            'trades': trades
        }

        return results

    def _get_state(self, idx):
        # Extract features for given timestep
        return self.data[idx]

    def _forecast_to_action(self, forecast):
        """Convert price forecast to trading action."""
        if forecast > 0.001:  # Buy if predicted return > 0.1%
            return 2
        elif forecast < -0.001:  # Sell if predicted return < -0.1%
            return 0
        else:
            return 1  # Hold

# Run backtest
backtester = Backtester(model, test_data)
results = backtester.run()

print("Backtest Results:")
for metric, value in results['metrics'].items():
    print(f"{metric}: {value:.2f}")
```

PART 8: HYPERPARAMETER TUNING

8.1 Hyperparameters to Tune

FORECASTING MODELS:

- `sequence_length`: 24, 48, 72, 168, 336 (hours)
- `hidden_size`: 32, 64, 128, 256
- `num_layers`: 1, 2, 3, 4
- `dropout`: 0.1, 0.2, 0.3, 0.5
- `learning_rate`: 1e-4, 3e-4, 1e-3, 3e-3
- `batch_size`: 32, 64, 128, 256
- `attention_heads`: 2, 4, 8

REINFORCEMENT LEARNING:

- `hidden_dim`: 64, 128, 256
- `learning_rate`: 1e-4, 3e-4, 1e-3
- `gamma` (discount factor): 0.95, 0.99, 0.999
- `epsilon_decay`: 0.99, 0.995, 0.999
- `buffer_size`: 10000, 50000, 100000
- `batch_size`: 32, 64, 128
- `update_frequency`: 1, 5, 10 (for target network)

8.2 Tuning Strategy

OPTION A: Grid Search (systematic but slow)

```python
```

```python
from itertools import product

param_grid = {
    'hidden_size': [64, 128],
    'num_layers': [2, 3],
    'learning_rate': [1e-3, 3e-3],
    'dropout': [0.1, 0.2]
}

best_score = float('inf')
best_params = None

for params in product(*param_grid.values()):
    param_dict = dict(zip(param_grid.keys(), params))

    # Train model with these params
    model = create_model(**param_dict)
    val_loss = train_and_evaluate(model, train_loader, val_loader)

    if val_loss < best_score:
        best_score = val_loss
        best_params = param_dict

print(f"Best params: {best_params}")
print(f"Best validation loss: {best_score}")
```

OPTION B: Random Search (faster, often as good)

```python
```

```python
from scipy.stats import uniform, randint

param_distributions = {
    'hidden_size': randint(32, 257),
    'num_layers': randint(1, 5),
    'learning_rate': uniform(1e-4, 1e-2),
    'dropout': uniform(0.0, 0.5)
}

n_iterations = 50
best_score = float('inf')

for i in range(n_iterations):
    params = {k: v.rvs() for k, v in param_distributions.items()}

    model = create_model(**params)
    val_loss = train_and_evaluate(model, train_loader, val_loader)

    if val_loss < best_score:
        best_score = val_loss
        best_params = params
```

OPTION C: Bayesian Optimization (most efficient)

python

```python
from bayes_opt import BayesianOptimization

def objective_function(hidden_size, num_layers, learning_rate, dropout):
    """Function to minimize - returns negative Sharpe ratio."""
    params = {
        'hidden_size': int(hidden_size),
        'num_layers': int(num_layers),
        'learning_rate': learning_rate,
        'dropout': dropout
    }

    model = create_model(**params)
    results = train_and_backtest(model)

    # Return negative Sharpe (because we maximize in Bayesian Opt)
    return -results['metrics']['Sharpe Ratio']

pbounds = {
    'hidden_size': (32, 256),
    'num_layers': (1, 4),
    'learning_rate': (1e-4, 1e-2),
    'dropout': (0.0, 0.5)
}

optimizer = BayesianOptimization(
    f=objective_function,
    pbounds=pbounds,
    random_state=42,
)

optimizer.maximize(init_points=10, n_iter=50)

print(f"Best parameters: {optimizer.max['params']}")
print(f"Best Sharpe: {-optimizer.max['target']:.2f}")
```

PART 9: PRODUCTION CONSIDERATIONS

9.1 Model Retraining Strategy

python

```python
# Option A: Periodic retraining (e.g., weekly)
def retrain_model_periodic(model, new_data, frequency='weekly'):
    """Retrain model on expanding window."""
    # Append new data to training set
    updated_train_data = np.concatenate([train_data, new_data])

    # Retrain
    model = train_model(updated_train_data)

    return model

# Option B: Performance-triggered retraining
def should_retrain(recent_performance, threshold_sharpe=1.0):
    """Retrain if performance drops below threshold."""
    if recent_performance['Sharpe Ratio'] < threshold_sharpe:
        return True
    return False

# Option C: Concept drift detection
from scipy.stats import ks_2samp

def detect_drift(recent_data, historical_data, significance=0.05):
    """Use Kolmogorov-Smirnov test to detect distribution shifts."""
    statistic, p_value = ks_2samp(historical_data, recent_data)
    return p_value < significance  # True if drift detected
```

9.2 Risk Management

python

```python
class RiskManager:
    def __init__(self, max_position_size=1.0, max_drawdown=0.20,
                 max_daily_loss=0.05, stop_loss=0.02):
        self.max_position_size = max_position_size
        self.max_drawdown = max_drawdown
        self.max_daily_loss = max_daily_loss
        self.stop_loss = stop_loss

        self.peak_value = 0
        self.daily_start_value = 0

    def check_constraints(self, portfolio_value, position, entry_price, current_pric
        """Return True if trade is allowed, False otherwise."""

        # Update peak
        self.peak_value = max(self.peak_value, portfolio_value)

        # Check max drawdown
        current_drawdown = (self.peak_value - portfolio_value) / self.peak_value
        if current_drawdown > self.max_drawdown:
            return False, "Max drawdown exceeded"

        # Check daily loss
        daily_loss = (self.daily_start_value - portfolio_value) / self.daily_start_v
        if daily_loss > self.max_daily_loss:
            return False, "Max daily loss exceeded"

        # Check stop loss on current position
        if position != 0:
            position_pnl = (current_price / entry_price - 1) * position
            if position_pnl < -self.stop_loss:
                return False, "Stop loss triggered"

        return True, "OK"

    def reset_daily(self, portfolio_value):
        """Call at start of each trading day."""
        self.daily_start_value = portfolio_value
```

PART 10: COMPLETE WORKFLOW SUMMARY

```
1. DATA PREPARATION
   ├── Load hourly OHLCV data
   ├── Handle missing values (forward fill, interpolation)
   ├── Calculate log returns (stationarity)
   └── Train/Val/Test split (60/20/20, sequential)

2. FEATURE ENGINEERING
   ├── Technical indicators (RSI, MACD, Bollinger Bands, ATR)
   ├── Lagged features (1h, 6h, 24h, 168h)
   ├── Rolling statistics (mean, std, skew, kurt)
   ├── Time-based features (cyclical encoding)
   ├── Cross-asset features (correlations, ranks)
   └── Feature selection (top 50-100 features)

3. NORMALIZATION
   ├── Rolling z-score (per asset)
   └── RobustScaler (fit on train only)

4. MODEL SELECTION
   ├── OPTION A: Temporal Fusion Transformer (forecasting)
   ├── OPTION B: Pyraformer/Informer (long sequences)
   ├── OPTION C: LSTM-Transformer hybrid
   └── OPTION D: Dueling DDQN / PPO (direct RL)
5. TRAINING
   ├── Define loss function (MSE + Directional for forecasting, PnL for RL)
   ├── Train with early stopping (patience=10)
   ├── Monitor validation loss
   └── Save best model

6. HYPERPARAMETER TUNING
   ├── Grid/Random/Bayesian optimization
   └── Optimize on validation Sharpe ratio

7. BACKTESTING
   ├── Run on held-out test data
   ├── Calculate metrics (Sharpe, Max DD, Win Rate)
   ├── Visualize equity curve
   └── Analyze trade distribution

8. RISK MANAGEMENT
   ├── Implement position limits
```

```
   ├── Set stop losses
   ├── Monitor drawdown
   └── Daily loss limits

9. DEPLOYMENT
   ├── Paper trading (forward testing)
   ├── Performance monitoring
   ├── Periodic retraining
   └── Drift detection
```

PART 11: IMPLEMENTATION ORDER (RECOMMENDED)

Phase 1: Baseline (Week 1-2)

```
1. Data cleaning & preprocessing
2. Simple technical indicators (5-10 features)
3. Train basic LSTM model
4. Backtest with simple buy/hold/sell rules
5. Establish baseline metrics (Sharpe ~0.5-1.0 expected)
```

Phase 2: Enhanced Features (Week 3)

```
1. Add comprehensive technical indicators (50+ features)
2. Feature selection to top 30-50
3. Improve LSTM or switch to Transformer
4. Target: Sharpe > 1.0
```

Phase 3: Advanced Models (Week 4-5)

```
1. Implement Temporal Fusion Transformer OR
2. Implement Dueling DDQN with experience replay
3. Hyperparameter tuning
4. Target: Sharpe > 1.5
```

Phase 4: Hybrid Approach (Week 6+)

```
1. Combine TFT predictions as RL state features
2. Train PPO agent using TFT forecasts
```

```
3. Optimize reward function
4. Target: Sharpe > 2.0
```

PART 12: COMMON PITFALLS & SOLUTIONS

Pitfall 1: Overfitting

SYMPTOMS:

- High training accuracy, poor test performance
- Perfect directional accuracy in validation, 50% in test

SOLUTIONS:

```python
# 1. Stronger regularization
model = PyraformerModel(
    input_dim=num_features,
    d_model=64,
    dropout=0.3,  # Increase from 0.1
    weight_decay=1e-4  # Add L2 regularization
)

# 2. Reduce model complexity
# Use fewer layers, smaller hidden size

# 3. More data augmentation
# Add noise, use multiple assets

# 4. Walk-forward validation
# Retrain on expanding windows
```

Pitfall 2: Look-Ahead Bias

SYMPTOMS:

- Suspiciously good results
- Performance degrades in live trading

SOLUTIONS:

```python
# ✅ Always verify features use only past data
# Check all .shift() operations are positive
# Check all rolling windows are computed backward-looking

# Example check:
assert df['rsi'].shift(1).notna().all()  # Ensure no future peeking
```

Pitfall 3: Transaction Costs Ignored

SYMPTOMS:

- High theoretical returns
- Excessive trading frequency

SOLUTIONS:

```python
# Include realistic costs in backtest
TRANSACTION_COST = 0.001  # 0.1% per trade (realistic for crypto)
SLIPPAGE = 0.0005  # 0.05% slippage

def calculate_net_return(gross_return, traded):
    if traded:
        return gross_return - TRANSACTION_COST - SLIPPAGE
    return gross_return
```

Pitfall 4: Insufficient Data

SYMPTOMS:

- Model can't learn patterns
- High variance in performance

SOLUTIONS:

```python
```

```
# 1. Use transfer learning
pretrained_model = load_pretrained_model('btc_predictor')
fine_tune_on_your_data(pretrained_model, your_data)

# 2. Data augmentation
# Add jittering, scaling, time-warping

# 3. Multi-asset learning
# Train on multiple correlated assets simultaneously
```

PART 13: SOTA BEST PRACTICES (2024-2026)

From Recent Literature

1. **Temporal Fusion Transformer performs best for multi-horizon forecasting** when integrating on-chain and technical indicators
2. **Pyraformer shows superior predictive accuracy and shorter training times** compared to traditional ARIMA and other transformer variants
3. **Dueling DDQN with prioritized experience replay outperforms DQN and DDQN** in empirical evaluations on recent market data (2024-2025)
4. **Hybrid approaches combining LSTM feature extraction with PPO learning** excel in high-return scenarios
5. **Cluster embedding for future trend prediction significantly improves DRL performance** over raw market data approaches
6. **Lag features, differencing, volatility measures, and momentum indicators** are essential feature engineering techniques
7. **Hybrid loss functions combining quantitative error and trend accuracy** optimize transformer models better than MSE alone

---

Recommended Configuration (2026 SOTA)

```
python
```

```
# FORECASTING STAGE
model_config = {
    'architecture': 'TemporalFusionTransformer',
    'encoder_length': 168,  # 1 week
    'prediction_length': 24,  # 1 day ahead
    'hidden_size': 64,
    'attention_heads': 4,
    'dropout': 0.2,
    'loss_function': 'HybridLoss(alpha=0.7)',  # MSE + Directional
}

# FEATURE SET
features = [
    'log_returns', 'volume',
    'rsi_14', 'macd', 'macd_signal',
    'bb_width', 'atr', 'adx',
    'ema_24', 'ema_168',
    'volatility_24h', 'volatility_168h',
    'obv', 'mfi',
    'return_lag_1', 'return_lag_24', 'return_lag_168',
    'hour_sin', 'hour_cos', 'dow_sin', 'dow_cos'
]  # 23 features total

# RL TRADING STAGE
rl_config = {
    'algorithm': 'DuelingDDQN',
    'state': features + ['balance', 'position', 'portfolio_value'],
    'action_space': 3,  # BUY, SELL, HOLD
    'hidden_dim': 128,
    'learning_rate': 3e-4,
    'gamma': 0.99,
    'epsilon_decay': 0.995,
    'buffer_size': 50000,
    'batch_size': 64,
    'reward_function': 'multi_objective',  # Profit + Risk + Cost
}

# TRAINING
training_config = {
    'train_pct': 0.6,
    'val_pct': 0.2,
    'test_pct': 0.2,
    'epochs': 100,
```

```
    'early_stopping_patience': 10,
    'learning_rate_scheduler': 'ReduceLROnPlateau',
}
```

PART 14: CODE ORGANIZATION

Recommended Project Structure

```
algorithmic_trading/
│
├── data/
│   ├── raw/              # Original data files
│   ├── processed/        # Cleaned & featured data
│   └── splits/           # Train/val/test splits
│
├── src/
│   ├── data/
│   │   ├── data_loader.py
│   │   ├── preprocessing.py
│   │   └── feature_engineering.py
│   │
│   ├── models/
│   │   ├── tft_model.py
│   │   ├── pyraformer.py
│   │   ├── lstm_transformer.py
│   │   ├── dqn_agent.py
│   │   └── ppo_agent.py
│   │
│   ├── training/
│   │   ├── train_forecaster.py
│   │   ├── train_rl.py
│   │   └── utils.py
│   │
│   ├── evaluation/
│   │   ├── metrics.py
│   │   ├── backtester.py
│   │   └── visualizations.py
│   │
│   └── trading/
│       ├── environment.py
│       ├── risk_manager.py
```

```
│       └── portfolio.py
│
├── notebooks/
│   ├── 01_eda.ipynb
│   ├── 02_feature_engineering.ipynb
│   └── 03_model_experiments.ipynb
│
├── configs/
│   ├── model_config.yaml
│   ├── training_config.yaml
│   └── backtest_config.yaml
│
├── tests/
│   └── test_models.py
│
├── requirements.txt
└── README.md
```

PART 15: DEPENDENCIES

txt

```
# Core
numpy==1.24.3
pandas==2.0.3
torch==2.0.1
scikit-learn==1.3.0

# Time Series
pytorch-forecasting==1.0.0
pytorch-lightning==2.0.4

# Technical Analysis
ta==0.11.0
ta-lib==0.4.28

# Visualization
matplotlib==3.7.2
seaborn==0.12.2
plotly==5.15.0

# Optimization
optuna==3.2.0
bayes-opt==1.4.3

# Reinforcement Learning
gym==0.26.2
stable-baselines3==2.0.0

# Utilities
tqdm==4.65.0
pyyaml==6.0.1
```

APPENDIX: QUICK REFERENCE

Key Papers (2024-2026)

- **Temporal Fusion Transformer approaches**: Recent studies from 2024-2025 demonstrate superior performance in crypto and equity markets using multi-source features
- **TradeAI Pyraformer study**: Shows best predictive accuracy on cryptocurrency data
- **Deep Q-Networks for portfolio optimization**: DQN, DDQN, and Dueling DDQN implementations on recent market data

- **Hybrid LSTM-PPO and Cluster Embedding-PPO**: State-of-the-art reinforcement learning frameworks

Most Important Takeaways

1. **Use Transformers for forecasting**, DRL for decision-making
2. **Feature engineering is critical** - technical indicators + lagged features
3. **Directional accuracy > RMSE** for trading success
4. **Walk-forward validation only** - no random splits
5. **Include transaction costs** in all backtests
6. **Monitor Sharpe ratio** as primary metric
7. **Implement risk management** from day 1
8. **Retrain periodically** on expanding windows
9. **Start simple, iterate** - don't overcomplicate initially
10. **Backtest thoroughly** before any live deployment

FINAL NOTES FOR LLM IMPLEMENTATION

This guide is structured for sequential implementation:

1. **Start with data preprocessing** (Part 2)
2. **Build feature pipeline** (Part 3)
3. **Implement baseline model** (Part 4.2 - simple Transformer)
4. **Add evaluation framework** (Part 7)
5. **Iterate with hyperparameter tuning** (Part 8)
6. **Deploy advanced models** (Part 4.1, 4.4 - TFT, DRL)
7. **Production considerations** (Part 9, 10)

Each code block is self-contained and can be adapted independently. Hyperparameters provided are starting points based on recent literature - tune for your specific data.

Expected Timeline:

- Baseline working model: 1-2 weeks
- Competitive model (Sharpe > 1.0): 3-4 weeks
- SOTA model (Sharpe > 1.5): 6-8 weeks

Success Metrics (Backtesting on Test Data):

- Baseline: Sharpe > 0.5, Win Rate > 45%
- Good: Sharpe > 1.0, Win Rate > 50%, Max DD < 20%
- Excellent: Sharpe > 1.5, Win Rate > 55%, Max DD < 15%
- Elite: Sharpe > 2.0, Win Rate > 60%, Max DD < 10%

Remember: **Past performance does not guarantee future results.** Always paper trade before risking real capital.